# Principal Component Analysis

## Projection, SVD, reconstruction, and modern applications

Kangning Cui
City University of Hong Kong (Dongguan)

Semester A, 2026/27

# Dimensionality motivation

CS5487 ▪ Semester A, 2026/27

# Dimensionality reduction

An ML system may collect many measurements that reflect only a few underlying factors.

- $d$: number of original features (e.g., $100$ sensor channels).
- $r$: number of retained coordinates, with $r < d$.
- PCA learns $r$ new coordinates from variation in the training data.

Example: $100$ correlated readings represented by $5$ principal coordinates.

$$\boxed{\boldsymbol{x} \in \mathbb{R}^{100} \longrightarrow \boldsymbol{z} \in \mathbb{R}^{5}}$$

**In practice:** a smaller input may reduce storage and speed up later models. Whether it preserves predictive information must still be checked.

# Redundant measurements

Two sensors respond to the same process at different scales.

| Sample | Sensor 1 | Sensor 2 |
|---|---|---|
| A | 1 | 2 |
| B | 2 | 4 |
| C | 3 | 6 |

**One underlying signal**

- Sensor 2 $= 2\times$ Sensor 1.
- Both measurements change together.
- One coordinate can encode their shared variation.

**Use:** compress redundant monitoring streams. Unexpected disagreement between sensors may appear as a reconstruction residual.

# High-dimensional coverage

Divide each of $d$ features into $10$ intervals. A uniform grid has $10^d$ cells. Suppose $1{,}000$ observations are spread evenly across that grid.

| Features $d$ | Grid cells | Average samples per cell |
|---|---|---|
| $2$ | $100$ | $10$ |
| $5$ | $100{,}000$ | $0.01$ |
| $10$ | $10^{10}$ | $10^{-7}$ |

- With fixed sample size, local neighborhoods become sparse.
- Density estimation or nearest-neighbor methods may need more data.

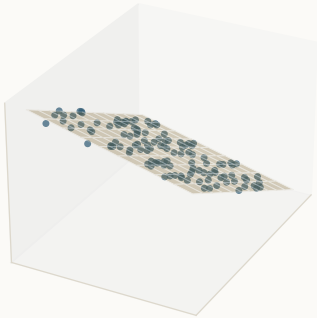
**Connection to PCA:** if data lie near a lower-dimensional linear structure, projecting onto it can remove redundant directions.

# Low-dimensional structure

**Many channels, few drivers**

- Spectral bands responding to a few material mixtures.
- Related sensors measuring one physical process.
- Images varying mainly with illumination.

**Use:** describe correlated measurements with fewer shared coordinates.



# PCA representation

For one observation $\boldsymbol{x} \in \mathbb{R}^d$, PCA seeks $r < d$ new coordinates $\boldsymbol{z} \in \mathbb{R}^r$.

$$\boxed{\boldsymbol{x} \xrightarrow{\text{encode}} \boldsymbol{z} \xrightarrow{\text{decode}} \hat{\boldsymbol{x}}}$$

- $\boldsymbol{z}$: compact coordinates describing important variation.
- $\hat{\boldsymbol{x}}$: approximate reconstruction in the original feature space.
- PCA learns directions from inputs alone, without target labels $y$.

**Example:** represent $50$ correlated sensor readings using $3$ PCA coordinates before transmission.

**PCA prioritizes large input variance. We must separately check whether discarded directions matter for the final task.**

# PCA geometry

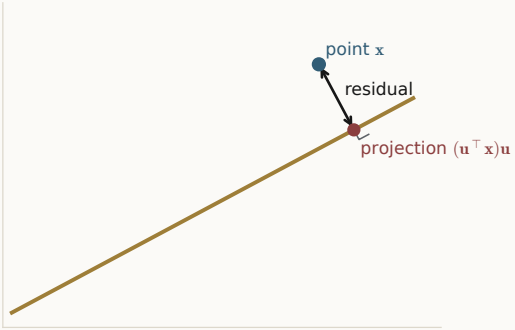
CS5487 ▪ Semester A, 2026/27

# Projection

Choose a **unit direction** $\boldsymbol{v} \in \mathbb{R}^d$, so $\|\boldsymbol{v}\|_2 = 1$. For a centered point $\tilde{\boldsymbol{x}}$, its coordinate along this direction is:

$$\boxed{z = \boldsymbol{v}^\mathsf{T} \tilde{\boldsymbol{x}}}$$

- $z$: scalar position along the direction.
- $z\boldsymbol{v}$: projected point.
- $\tilde{\boldsymbol{x}} - z\boldsymbol{v}$: perpendicular residual.



# Unit-length constraint

Scaling a direction changes its dot product even when the line stays the same. For the point $\boldsymbol{x} = (3, 4)^\mathsf{T}$:

$$(1, 0) \cdot (3, 4) = 3, \qquad (2, 0) \cdot (3, 4) = 6.$$

The vectors $(1, 0)^\mathsf{T}$ and $(2, 0)^\mathsf{T}$ point in the same direction.

- The second score is twice as large only because the vector is longer.
- Requiring $\|\boldsymbol{v}\|_2 = 1$ makes the scalar projection comparable across directions.

Direction, not arbitrary vector length, should determine PCA.

# Projection example

For the centered observation and diagonal unit direction:

$$\tilde{\boldsymbol{x}} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \qquad \boldsymbol{v} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

1. **Encode** as one coordinate:

$$z = \boldsymbol{v}^{\mathsf{T}} \tilde{\boldsymbol{x}} = \sqrt{2}.$$

2. **Project and find the residual**:

$$z\boldsymbol{v} = \boxed{\begin{bmatrix} 1 \\ 1 \end{bmatrix}}, \qquad \tilde{\boldsymbol{x}} - z\boldsymbol{v} = \boxed{\begin{bmatrix} 1 \\ -1 \end{bmatrix}}.$$

**Meaning:** for two correlated sensor channels, the projection captures their common movement; the residual captures their disagreement.

# Centering

PCA measures variation **around the sample mean**. For $n$ observations $\boldsymbol{x}_i \in \mathbb{R}^d$:

$$\bar{\boldsymbol{x}} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{x}_i, \qquad \tilde{\boldsymbol{x}}_i = \boldsymbol{x}_i - \bar{\boldsymbol{x}}.$$

- $\bar{\boldsymbol{x}}$: mean feature vector.
- $\tilde{\boldsymbol{x}}_i$: centered observation.

Define the covariance matrix:

$$\boxed{\boldsymbol{S} = \frac{1}{n}\sum_{i=1}^{n} \tilde{\boldsymbol{x}}_i \tilde{\boldsymbol{x}}_i^{\mathsf{T}}}.$$

We consistently use the $1/n$ convention for PCA in this lecture.

# Centering example

Consider three two-feature observations:

$$\boldsymbol{x}_1 = (1,1)^\mathsf{T}, \quad \boldsymbol{x}_2 = (3,3)^\mathsf{T}, \quad \boldsymbol{x}_3 = (5,5)^\mathsf{T}.$$

Their mean is $\bar{\boldsymbol{x}} = (3,3)^\mathsf{T}$.

| Original observation | Centered observation |
|---|---|
| $(1,1)$ | $(-2,-2)$ |
| $(3,3)$ | $(0,0)$ |
| $(5,5)$ | $(2,2)$ |

**Meaning:** PCA studies how readings vary around their baseline, not the arbitrary baseline value itself.

# Effect of centering

Two datasets can have the same variation at different locations.

| Observed values | Mean | Centered values |
|---|---|---|
| $1, 2, 3$ | $2$ | $-1, 0, 1$ |
| $11, 12, 13$ | $12$ | $-1, 0, 1$ |

Adding a constant $\boldsymbol{c}$ shifts both the points and their mean by $\boldsymbol{c}$:

$$(\boldsymbol{x}_i + \boldsymbol{c}) - (\bar{\boldsymbol{x}} + \boldsymbol{c}) = \boldsymbol{x}_i - \bar{\boldsymbol{x}}.$$

**Centering makes PCA depend on the shape of the data, not its location.**

# Covariance example

Use the centered observations $(-2,-2)$, $(0,0)$ and $(2,2)$.

$$\boldsymbol{S} = \frac{1}{3}\left(\begin{bmatrix}4 & 4\\ 4 & 4\end{bmatrix} + \begin{bmatrix}0 & 0\\ 0 & 0\end{bmatrix} + \begin{bmatrix}4 & 4\\ 4 & 4\end{bmatrix}\right).$$

Thus:

$$\boxed{\boldsymbol{S} = \begin{bmatrix}8/3 & 8/3\\ 8/3 & 8/3\end{bmatrix}}.$$

- Diagonal entries: individual feature variances.
- Off-diagonal entries: how the features vary together.

**Use:** these perfectly correlated readings lie on one line, so one principal coordinate can reconstruct them without loss.

# Feature scaling

PCA maximizes **numerical** variance, so measurement units can change its result.

**Keep original units**

- Comparable physical measurements.
- Absolute variation matters.

**Standardize first**

- Different or arbitrary units.
- Equalize each feature's scale.

For feature $j$, standardization uses $(x_j - \bar{x}_j)/s_j$, where $s_j$ is its training standard deviation.

**Scaling is a modeling choice; do not apply it automatically.**

# Units change variance

Suppose heights measured in meters have numerical variance $0.01$. After converting to centimeters, $h_{\mathrm{cm}} = 100 h_{\mathrm{m}}$:

$$\mathrm{Var}(h_{\mathrm{cm}}) = 100^2 (0.01) = \boxed{100}.$$

- The people and physical differences did not change.
- Numerical variance increased $10{,}000$-fold solely from units.
- Raw-covariance PCA may now favor height over another feature.

**Practical decision:** standardize if centimeters, dollars, or other arbitrary units should not determine which feature dominates.

# First principal component

For each centered observation $\tilde{\boldsymbol{x}}_i$, choose a unit direction $\boldsymbol{v}$ and compute:

$$\boxed{z_i = \boldsymbol{v}^\mathsf{T} \tilde{\boldsymbol{x}}_i}.$$

These $n$ scalar coordinates form the one-dimensional projection of the dataset.

- If the $z_i$ values spread out, the direction retains substantial variation.
- If they nearly coincide, the direction reveals little variation.

**The first principal component (PC1) maximizes the variance of $z_1, \ldots, z_n$.**

# Projection direction comparison

Project centered sensor readings $(-2,-2)$, $(0,0)$ and $(2,2)$ onto two unit directions.

| Direction $\boldsymbol{v}$ | Coordinates $z_i$ | Variance $\frac{1}{3}\sum_i z_i^2$ |
|---|---|---|
| $(1,0)$ | $(-2,0,2)$ | $8/3$ |
| $(1,1)/\sqrt{2}$ | $(-2\sqrt{2},0,2\sqrt{2})$ | $16/3$ |

**Result:** the diagonal direction captures twice as much variance. **Meaning:** one principal coordinate can follow the shared signal of both sensors, rather than preserving only one sensor's reading.

# Projected variance

For centered data, the projected coordinate is $z_i = \boldsymbol{v}^\mathsf{T} \tilde{\boldsymbol{x}}_i$. Its mean is zero, so its variance is:

$$\begin{aligned} \mathrm{Var}(z) &= \frac{1}{n} \sum_i z_i^2 \\ &= \boldsymbol{v}^\mathsf{T} \left( \frac{1}{n} \sum_i \tilde{\boldsymbol{x}}_i \tilde{\boldsymbol{x}}_i^\mathsf{T} \right) \boldsymbol{v}. \end{aligned}$$

The term in parentheses is $\boldsymbol{S}$, the covariance matrix.

$$\boxed{\mathrm{Var}(z) = \boldsymbol{v}^\mathsf{T} \boldsymbol{S} \boldsymbol{v}}.$$

This converts a geometric choice of direction into a mathematical objective.

# Maximum variance

We want the unit direction $\boldsymbol{v}$ with the largest projected variance:

$$\max_{\boldsymbol{v}} \ \boldsymbol{v}^{\mathsf{T}} \boldsymbol{S} \boldsymbol{v} \quad \text{subject to} \quad \boldsymbol{v}^{\mathsf{T}} \boldsymbol{v} = 1.$$

- Objective: keep as much data variation as possible.
- Constraint: compare directions at the same vector length.

Without the constraint, using $10\boldsymbol{v}$ instead of $\boldsymbol{v}$ would multiply the objective by $100$ without changing the direction.

# Constrained PCA derivation

**Goal:** maximize projected variance without artificially increasing the length of $\boldsymbol{v}$:

$$\max_{\boldsymbol{v}} \ \boldsymbol{v}^{\mathsf{T}} \boldsymbol{S} \boldsymbol{v} \quad \text{subject to} \quad \boldsymbol{v}^{\mathsf{T}} \boldsymbol{v} = 1.$$

1. **Include the constraint.** A Lagrange multiplier $\lambda$ enforces unit length:

$$\mathcal{J}(\boldsymbol{v}, \lambda) = \boldsymbol{v}^{\mathsf{T}} \boldsymbol{S} \boldsymbol{v} - \lambda(\boldsymbol{v}^{\mathsf{T}} \boldsymbol{v} - 1).$$

2. **Find a stationary direction.** Since the covariance $\boldsymbol{S}$ is symmetric:

$$\nabla_{\boldsymbol{v}} \mathcal{J} = 2\boldsymbol{S}\boldsymbol{v} - 2\lambda \boldsymbol{v} = \boldsymbol{0} \quad \Longrightarrow \quad \boxed{\boldsymbol{S}\boldsymbol{v} = \lambda \boldsymbol{v}}.$$

**Meaning:** the optimal direction must be a covariance eigenvector. We choose the one with the largest eigenvalue (explained next).

# Eigenvalue interpretation

The previous derivation gave the covariance eigenvalue equation:

$$\boldsymbol{S}\boldsymbol{v} = \lambda \boldsymbol{v}.$$

For a unit eigenvector $\boldsymbol{v}$, projected variance becomes:

$$\mathrm{Var}(z) = \boldsymbol{v}^{\mathsf{T}} \boldsymbol{S} \boldsymbol{v} = \lambda \underbrace{\boldsymbol{v}^{\mathsf{T}} \boldsymbol{v}}_{1} = \boxed{\lambda}.$$

- Eigenvector $\boldsymbol{v}$: candidate projection direction.
- Eigenvalue $\lambda$: variance retained along that direction.
- Largest eigenvalue: first principal component (PC1).

**Use:** compute principal directions directly from the data covariance, then rank them by captured variance.

# Multiple components

Order covariance eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d$ and keep their first $r$ unit eigenvectors:

$$\boldsymbol{V}_r = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_r] \in \mathbb{R}^{d \times r}.$$

The directions are orthonormal, so $\boldsymbol{V}_r^\mathsf{T} \boldsymbol{V}_r = \boldsymbol{I}_r$. For centered observation $\tilde{\boldsymbol{x}}_i$:

$$\boxed{\boldsymbol{z}_i = \boldsymbol{V}_r^\mathsf{T} \tilde{\boldsymbol{x}}_i \in \mathbb{R}^r}.$$

- PC1 captures the most variance; later PCs capture remaining orthogonal variation.
- Each observation now has only $r$ numbers for downstream learning.

Example: $d = 100$ spectral bands $\rightarrow$ $r = 10$ PCA features for a smaller classifier.

# Two-dimensional PCA

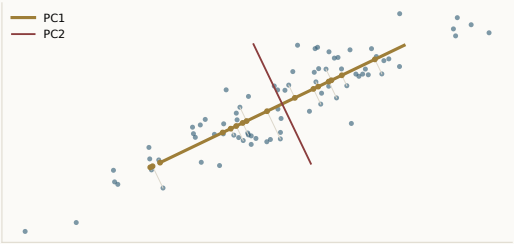
Consider the covariance matrix:

$$\boldsymbol{S} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}.$$

It has eigenvalues $\lambda_1 = 5$, $\lambda_2 = 1$ and unit eigenvectors:

$$\boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \boldsymbol{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

PC1 follows the cloud's longer axis.



# Comparing PCA directions

For $\boldsymbol{S} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$, compare the projected variances of three unit directions.

| Direction $\boldsymbol{v}$ | $\boldsymbol{v}^\top \boldsymbol{S} \boldsymbol{v}$ |
|---|---|
| Horizontal $(1, 0)$ | $3$ |
| Diagonal $(1, 1)/\sqrt{2}$ | $5$ |
| Other diagonal $(1, -1)/\sqrt{2}$ | $1$ |

PC1 is the diagonal direction, with variance $5$.

The eigenvalue gives its captured variance without projecting every data point explicitly.

# Explained variance

For covariance $\boldsymbol{S} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$, the eigenvalues are $5$ and $1$. Total variance is $5 + 1 = 6$.

$$\mathrm{EVR}(1) = \frac{5}{6} = \boxed{83.3\%}, \qquad \mathrm{EVR}(2) = \frac{6}{6} = \boxed{100\%}.$$

Here $\mathrm{EVR}(r)$ is the fraction of total variance retained by $r$ principal components.

- $r = 1$: one stored coordinate per observation, retaining $83.3\%$ of variation.
- $r = 2$: no dimensionality reduction and all variation retained.

**Use:** EVR quantifies the compression–information trade-off, not the accuracy of a later classifier.

# Three-dimensional variance

Suppose three features have covariance eigenvalues $7$, $2$ and $1$.

| Retained PCs $r$ | Captured variance | Cumulative EVR |
|---|---|---|
| $1$ | $7$ | $70\%$ |
| $2$ | $7+2=9$ | $90\%$ |
| $3$ | $10$ | $100\%$ |

- A $90\%$ target allows compression from three coordinates to two.
- A $95\%$ target requires all three, so no compression occurs.

**Practical lesson:** choose a threshold that fits the storage, reconstruction, or prediction requirement.

# Reconstruction view

CS5487 ▪ Semester A, 2026/27

# Encode and reconstruct

After fitting $r$ principal directions, store them in $\boldsymbol{V}_r$.

**Encode ($d \to r$)**

$$\boldsymbol{z}_i = \boldsymbol{V}_r^{\mathsf{T}}(\boldsymbol{x}_i - \bar{\boldsymbol{x}}).$$

**Decode ($r \to d$)**

$$\hat{\boldsymbol{x}}_i = \bar{\boldsymbol{x}} + \boldsymbol{V}_r \boldsymbol{z}_i.$$

Here $\hat{\boldsymbol{x}}_i$ is the reconstruction; substituting the encoding gives:

$$\boxed{\hat{\boldsymbol{x}}_i = \bar{\boldsymbol{x}} + \boldsymbol{V}_r \boldsymbol{V}_r^{\mathsf{T}}(\boldsymbol{x}_i - \bar{\boldsymbol{x}})}.$$

**Use:** store or transmit short $\boldsymbol{z}_i$ vectors while sharing one learned mean and PCA basis.

# Reconstruction example

Assume two centered sensor readings usually move together. For:

$$\tilde{\boldsymbol{x}} = (3,1)^{\mathsf{T}}, \qquad \boldsymbol{v} = \frac{1}{\sqrt{2}}(1,1)^{\mathsf{T}},$$

**Encode one coordinate**:

$$z = \boldsymbol{v}^{\mathsf{T}}\tilde{\boldsymbol{x}} = \frac{3+1}{\sqrt{2}} = 2\sqrt{2}.$$

**Reconstruct in two dimensions**:

$$\hat{\tilde{\boldsymbol{x}}} = z\boldsymbol{v} = \boxed{(2,2)^{\mathsf{T}}}.$$

The residual is $(1,-1)^{\mathsf{T}}$. **Meaning:** PCA keeps the common change and loses the difference between the two sensors. That difference may be measurement noise or a meaningful fault.

# Projection and residual

Every centered point splits into an orthogonal projection and a residual:

$$\tilde{\boldsymbol{x}} = \underbrace{\boldsymbol{v}\boldsymbol{v}^{\mathsf{T}}\tilde{\boldsymbol{x}}}_{\text{projection}} + \underbrace{(\tilde{\boldsymbol{x}} - \boldsymbol{v}\boldsymbol{v}^{\mathsf{T}}\tilde{\boldsymbol{x}})}_{\text{residual}}.$$

The two parts are perpendicular. By Pythagoras:

$$\boxed{\|\tilde{\boldsymbol{x}}\|_2^2 = \|\text{projection}\|_2^2 + \|\text{residual}\|_2^2}.$$

For a fixed point, capturing more squared length in the projection leaves less reconstruction error.

# Pythagoras in numbers

Return to $\tilde{x} = (3, 1)^{\mathsf{T}}$ projected onto the diagonal unit direction.

| Vector | Squared length |
|---|---|
| Original $(3, 1)$ | $3^2 + 1^2 = 10$ |
| Projection $(2, 2)$ | $2^2 + 2^2 = 8$ |
| Residual $(1, -1)$ | $1^2 + (-1)^2 = 2$ |

$$\boxed{10 = 8 + 2}.$$

**Meaning:** this individual observation loses squared error $2$ when stored in one dimension. Dataset-level explained variance, however, is calculated over *all* observations.

# Reconstruction error

For one unit direction $\boldsymbol{v}$, define average **squared reconstruction error**:

$$E(\boldsymbol{v}) = \frac{1}{n} \sum_i \left\| \tilde{\boldsymbol{x}}_i - \boldsymbol{v}\boldsymbol{v}^\mathsf{T} \tilde{\boldsymbol{x}}_i \right\|_2^2 .$$

By the orthogonal decomposition:

$$E(\boldsymbol{v}) = \underbrace{\frac{1}{n} \sum_i \|\tilde{\boldsymbol{x}}_i\|_2^2}_{\text{fixed total variation}} - \underbrace{\boldsymbol{v}^\mathsf{T} \boldsymbol{S} \boldsymbol{v}}_{\text{captured variance}} .$$

Therefore:

$$\boxed{\min E(\boldsymbol{v}) \iff \max \mathrm{Var}(z)}.$$

**Use:** the maximum-variance direction is also the best one-dimensional linear reconstruction, measured by average squared error.

# Discarded variance

Order covariance eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$. Keeping the first $r$ directions gives:

**Captured variance**

$$\sum_{j=1}^{r} \lambda_j.$$

**Average squared error**

$$\boxed{\sum_{j=r+1}^{d} \lambda_j}.$$

**Example:** eigenvalues $(5, 1)$, keeping $r = 1$ retains $5/6$ of variance but leaves mean squared reconstruction error $1$ per observation. **Use:** select a compression level that meets an acceptable reconstruction-error budget; the error is measured in squared feature units.

# Selecting the dimension

The **cumulative explained variance ratio** after keeping $r$ components is:

$$\boxed{\mathrm{EVR}(r) = \frac{\sum_{j=1}^{r} \lambda_j}{\sum_{j=1}^{d} \lambda_j}}.$$

Choose $r$ according to your task:

- **Compression:** preserve a target fraction (e.g., $95\%$).
- **Computation:** keep within a memory or runtime budget.
- **Prediction:** compare several $r$ values on validation data.

A higher variance ratio is not automatically a better predictive representation.

# Component selection exercise

The covariance matrix of a three-feature dataset has eigenvalues:

$$\lambda_1 = 8, \qquad \lambda_2 = 1, \qquad \lambda_3 = 1.$$

1. What is the smallest $r$ retaining at least $90\%$ of the total variance?
2. What is the average squared reconstruction error when retaining that $r$?

Use the sum of retained eigenvalues for explained variance and the sum of discarded eigenvalues for reconstruction error.

# Component selection solution

The total covariance variance is $8 + 1 + 1 = 10$.

| Retained PCs | Cumulative explained variance |
|---|---|
| 1 | $8/10 = 80\%$ |
| 2 | $(8 + 1)/10 = 90\%$ |

**Answer 1:** $\boxed{r = 2}$ is the smallest dimension meeting $90\%$ EVR. **Answer 2:** the discarded eigenvalue is $\lambda_3 = 1$, so average squared reconstruction error is $\boxed{1}$. **Decision:** reduce three sensor features to two if that loss is acceptable; otherwise retain more components.

# Variance and predictive value

PCA uses feature observations $\boldsymbol{x}$, not class labels $y$.

**High-variance direction**

- Kept early by PCA.
- May reflect irrelevant lighting or background.

**Low-variance direction**

- Easy for PCA to discard.
- May separate the classes.

## Common mistake

High explained variance does not guarantee useful class separation.

# Predictive low-variance signal

Consider two uncorrelated features for a classification task.

| Feature | Variance | Interpretation |
|---|---|---|
| $x_1$ | $25$ | Background/illumination variation |
| $x_2$ | $0.25$ | Small but class-related signal |

One-component PCA keeps $x_1$ and explains:

$$\frac{25}{25 + 0.25} \approx \boxed{99.0\%}$$

of total variance, yet it may discard the feature that distinguishes the classes. **Practice:** compare classifier validation performance before and after PCA, not just EVR.

# Uncorrelated coordinates

Using all principal directions, $\boldsymbol{V} = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_d]$, PCA coordinates are:

$$\boldsymbol{z} = \boldsymbol{V}^\mathsf{T}(\boldsymbol{x} - \boldsymbol{\mu}),$$

where $\boldsymbol{\mu}$ is the training feature mean. Their covariance is:

$$\mathrm{Cov}(\boldsymbol{z}) = \boldsymbol{V}^\mathsf{T}\boldsymbol{S}\boldsymbol{V} = \boxed{\mathrm{diag}(\lambda_1, \ldots, \lambda_d)}.$$

- Off-diagonal covariance becomes zero: PCs are uncorrelated.
- This summarizes redundancy and can simplify covariance analysis.

**Limit:** uncorrelated coordinates are not necessarily statistically independent.

# Uncorrelated and independent

**Uncorrelated**

- Covariance is zero.
- What PCA guarantees.

**Independent**

- Knowing one gives no information about the other.
- A stronger condition.

Example: let $u$ be equally likely to be $-1$, $0$ or $1$. Then $u$ and $u^2$ have zero covariance, but knowing $u = 0$ tells us $u^2 = 0$.

uncorrelated does not always mean independent.

# Singular value decomposition

CS5487 ▪ Semester A, 2026/27

# Centered data matrix

Store $n$ centered observations as rows, each with $d$ features:

$$\boldsymbol{X}_c = \begin{bmatrix} \tilde{\boldsymbol{x}}_1^\mathsf{T} \\ \vdots \\ \tilde{\boldsymbol{x}}_n^\mathsf{T} \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

The covariance is:

$$\boxed{\boldsymbol{S} = \frac{1}{n} \boldsymbol{X}_c^\mathsf{T} \boldsymbol{X}_c}.$$

**Computational point:** for $d = 10{,}000$ features, an explicit covariance matrix has $10^8$ entries. SVD can work directly on the centered data matrix instead.

# Small data matrix

Consider four centered observations with two features:

$$\boldsymbol{X}_c = \begin{bmatrix} -2 & 0 \\ 0 & -1 \\ 0 & 1 \\ 2 & 0 \end{bmatrix}, \qquad n = 4.$$

Compute the feature cross-product matrix:

$$\boldsymbol{X}_c^{\mathsf{T}} \boldsymbol{X}_c = \begin{bmatrix} 8 & 0 \\ 0 & 2 \end{bmatrix}.$$

Divide by $n$ to obtain the covariance:

$$\boxed{\boldsymbol{S} = \frac{1}{4} \boldsymbol{X}_c^{\mathsf{T}} \boldsymbol{X}_c = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix}}.$$

The first feature carries four times as much variance as the second.

# SVD

For centered $\boldsymbol{X}_c \in \mathbb{R}^{n \times d}$, let $q = \operatorname{rank}(\boldsymbol{X}_c)$. The **compact singular value decomposition (SVD)** is:

$$\boxed{\boldsymbol{X}_c = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^{\mathsf{T}}}.$$

| Matrix | Dimensions | Meaning |
|---|---|---|
| $\boldsymbol{U}$ | $n \times q$ | Sample-side directions |
| $\boldsymbol{D}$ | $q \times q$ | Diagonal singular values $s_j > 0$ |
| $\boldsymbol{V}$ | $d \times q$ | Feature-space directions |

Columns of $\boldsymbol{U}$ and $\boldsymbol{V}$ are orthonormal.

# PCA from SVD

Start from $\boldsymbol{S} = \boldsymbol{X}_c^\mathsf{T}\boldsymbol{X}_c/n$ and insert the SVD:

$$\begin{aligned}\boldsymbol{S} &= \frac{1}{n}(\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^\mathsf{T})^\mathsf{T}(\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^\mathsf{T}) \\ &= \boldsymbol{V}\frac{\boldsymbol{D}^2}{n}\boldsymbol{V}^\mathsf{T},\end{aligned}$$

using $\boldsymbol{U}^\mathsf{T}\boldsymbol{U} = \boldsymbol{I}$. Therefore:

$$\boxed{\lambda_j = \frac{s_j^2}{n}}, \qquad \boxed{\boldsymbol{v}_j = \text{column } j \text{ of } \boldsymbol{V}}.$$

SVD and covariance eigendecomposition recover the same nonzero PCA directions and variances.

# SVD numerical result

For the four-observation example, $\boldsymbol{X}_c^{\mathsf{T}}\boldsymbol{X}_c = \operatorname{diag}(8, 2)$.

| Component | Singular value $s_j$ | Eigenvalue $s_j^2/4$ |
|---|---|---|
| $1$ | $\sqrt{8} = 2\sqrt{2}$ | $2$ |
| $2$ | $\sqrt{2}$ | $0.5$ |

$$\boxed{\mathrm{EVR}(1) = \frac{2}{2 + 0.5} = 80\%}.$$

**Meaning:** one principal coordinate retains $80\%$ of the observed variation. If the lost $20\%$ matters to the task, keep both coordinates.

# SVD coordinates

For $n$ observations stored as rows and $d$ measured features as columns, use $\boldsymbol{X}_c = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^\mathsf{T}$.

$$\boxed{\boldsymbol{X}_c\boldsymbol{V} = \boldsymbol{U}\boldsymbol{D}}.$$

- Columns of $\boldsymbol{V} \in \mathbb{R}^{d\times q}$: PCA directions in feature space.
- Rows of $\boldsymbol{U}\boldsymbol{D}$: PCA coordinates of the $n$ training observations.
- Singular values in $\boldsymbol{D}$: strengths of the principal directions.

**For a new observation** $\boldsymbol{x}_*$, reuse the fitted mean and first $r$ directions:

$$\boxed{\boldsymbol{z}_* = \boldsymbol{V}_r^\mathsf{T}(\boldsymbol{x}_* - \bar{\boldsymbol{x}})}.$$

**Practical meaning:** SVD supplies both the projection directions and the training coordinates; new samples use $\boldsymbol{V}_r$, not rows of $\boldsymbol{U}$.

# Low-rank approximation

Keep the largest $r$ singular values and vectors:

$$\boxed{\boldsymbol{X}_{c,r} = \boldsymbol{U}_r \boldsymbol{D}_r \boldsymbol{V}_r^{\mathsf{T}}}.$$

The **Eckart–Young theorem** states that this is the best rank-$r$ approximation for squared Frobenius error:

$$\boxed{\|\boldsymbol{X}_c - \boldsymbol{X}_{c,r}\|_F^2 = \sum_{j>r} s_j^2}.$$

The Frobenius norm squares and sums *all matrix entries*; divide by $n$ for average squared error per observation. **Meaning:** no other rank-$r$ matrix preserves the training data better under this specific error measure; downstream prediction may prefer something else.

# Rank-one SVD example

Retain the first singular direction of our centered data matrix:

$$\boldsymbol{X}_c = \begin{bmatrix} -2 & 0 \\ 0 & -1 \\ 0 & 1 \\ 2 & 0 \end{bmatrix}, \quad \boldsymbol{X}_{c,1} = \begin{bmatrix} -2 & 0 \\ 0 & 0 \\ 0 & 0 \\ 2 & 0 \end{bmatrix}.$$

The discarded feature values are $-1$ and $1$:

$$\underbrace{\|\boldsymbol{X}_c - \boldsymbol{X}_{c,1}\|_F^2}_{\text{total error}} = 1^2 + 1^2 = \boxed{2}.$$

Average squared error is $2/4 = \boxed{0.5} = \lambda_2$. **Practical trade-off:** one feature has been compressed away. This is harmless if it is noise, but risky if it carries an important signal.

# Applications

CS5487 ▪ Semester A, 2026/27

# Compression

Suppose one observation contains $d = 1{,}000$ measured features. If PCA retains $r = 20$ principal components, its coordinate vector has only $20$ entries:

$$\boxed{\boldsymbol{x} \in \mathbb{R}^{1000} \longrightarrow \boldsymbol{z} \in \mathbb{R}^{20}}.$$

- $50$ times fewer coordinates per observation.
- Lower input dimension for some downstream algorithms.
- Approximate reconstruction rather than exact recovery.

A complete compression system must also store the PCA mean and the $1{,}000 \times 20$ basis matrix.

# Storage example

Store $n = 10{,}000$ observations with $d = 1{,}000$ features using $r = 20$ PCA components. Assume $4$ bytes per value (float32).

| Stored item | Values | Decimal size |
|---|---|---|
| Original observations | $10{,}000 \times 1{,}000$ | $40$ MB |
| PCA coordinates | $10{,}000 \times 20$ | $0.800$ MB |
| Shared basis | $1{,}000 \times 20$ | $0.080$ MB |
| Training mean | $1{,}000$ | $0.004$ MB |

$$\boxed{40 \text{ MB} \longrightarrow 0.884 \text{ MB} \ (\approx 45\times \text{ smaller})}.$$

**Use:** reduce stored or transmitted sensor data. Sizes exclude file overhead and temporary PCA training memory.

# Denoising

**PCA denoising**

1. Fit on representative training observations.
2. Keep the leading directions.
3. Reconstruct without discarded variation.

**Example:** centered readings $(2.2, 1.8)$ project to $(2, 2)$ along the diagonal, removing their disagreement.




## Common mistake

Denoising helps only when discarded variation is mainly noise, not useful signal.

# Residuals

The PCA reconstruction residual of a new observation is:

$$\boxed{\boldsymbol{r} = \boldsymbol{x} - \hat{\boldsymbol{x}}}, \qquad \|\boldsymbol{r}\|_2^2 \text{ is its squared reconstruction error.}$$

- Small residual: the point is close to the learned PCA subspace.
- Large residual: the point has unusual variation outside that subspace.

**Possible use:** monitor an industrial sensor system trained on normal operation. A large residual can flag readings for inspection. **Caution:** a large residual is not automatically a fault.

# Residual as a warning

Retain the unit direction $\boldsymbol{v} = (1, 1)^{\mathsf{T}}/\sqrt{2}$.

| Centered reading | Reconstruction | Squared residual |
|---|---|---|
| $(2, 2)$ | $(2, 2)$ | $0$ |
| $(3, 1)$ | $(2, 2)$ | $1^2 + (-1)^2 = 2$ |

The second reading deviates more from the learned shared-signal pattern.

- In condition monitoring, use normal validation data to choose an alert threshold.
- Investigate alarms before deciding that a fault occurred.

# Visualization

Reduce high-dimensional observations to two principal coordinates for plotting:

$$\boldsymbol{x} \in \mathbb{R}^{100} \longrightarrow (z_1, z_2) \in \mathbb{R}^2.$$

**Example:** plot samples with $100$ spectral measurements and color the dots by collection day.

- A shift in plotted locations may suggest a measurement or batch effect.
- Tight groups can suggest structure worth investigating.
- Overlap does not prove the labels are indistinguishable.

**Limit:** a 2D PCA plot preserves selected variance, not necessarily class separation or all original distances.

# Downstream models

Use PCA as one fitted stage of a prediction pipeline:

$$\boxed{\text{raw features} \;\rightarrow\; \text{PCA coordinates} \;\rightarrow\; \text{classifier}}.$$

**Illustrative validation results** (not measurements from a real experiment):

| Classifier input | Validation accuracy |
|---|---|
| All $100$ features | $90\%$ |
| PCA with $r = 5$ | $86\%$ |
| PCA with $r = 20$ | $92\%$ |

**Lesson:** choose $r$ using the downstream validation metric and computational cost. High explained variance alone cannot select the best predictor.

# Related methods

CS5487 ▪ Semester A, 2026/27

# Truncated SVD

**PCA** uses SVD on centered data $\boldsymbol{X}_c$; **truncated SVD** can approximate an uncentered matrix $\boldsymbol{X}$.

**Centered PCA**

- Subtract feature means.
- Preserve covariance variation.

**Uncentered truncated SVD**

- Often preserves sparse text inputs.
- Approximate the term–document matrix directly.

**Example:** compress a $1{,}000 \times 5{,}000$ document–word matrix to rank $50$; the coordinates can support document retrieval by latent patterns (latent semantic analysis).

# Probabilistic PCA

**Motivation:** measured observations lie near a low-dimensional subspace but also contain random noise. Probabilistic PCA writes:

$$\boxed{\boldsymbol{x} = \boldsymbol{\mu} + \boldsymbol{W}\boldsymbol{z} + \boldsymbol{\epsilon}}.$$

- $\boldsymbol{z} \in \mathbb{R}^r$: hidden factors describing shared variation.
- $\boldsymbol{W} \in \mathbb{R}^{d \times r}$: maps factors to the observed features.
- $\boldsymbol{\epsilon}$: Gaussian measurement noise; standard PPCA uses equal variance in all directions.

**Use:** interpret correlated noisy sensor measurements through latent factors and explicitly represent observation uncertainty.

# Factor analysis

Factor analysis uses the same general latent model:

$$\boldsymbol{x} = \boldsymbol{\mu} + \boldsymbol{W}\boldsymbol{z} + \boldsymbol{\epsilon}.$$

It changes the **noise assumption**:

- Probabilistic PCA: one common observation-noise variance.
- Factor analysis: each feature may have its own noise variance.

**Example:** two sensors measure one hidden process, but their measurement-noise variances are $0.1$ and $1.0$. **Use:** separate shared variation from feature-specific sensor reliability, rather than treating all measurements as equally noisy.

# Autoencoders

Both PCA and an autoencoder encode observations into fewer coordinates and reconstruct them.

**PCA: linear**

$$\boldsymbol{z} = \boldsymbol{V}_r^{\mathsf{T}}(\boldsymbol{x} - \boldsymbol{\mu}).$$

- SVD/eigenvector solution.
- Best linear squared-error subspace.

**Autoencoder: learned maps**

$$\boldsymbol{z} = f_\theta(\boldsymbol{x}), \quad \hat{\boldsymbol{x}} = g_\phi(\boldsymbol{z}).$$

- Often nonlinear.
- Fitted using reconstruction loss.

Here $\theta$ and $\phi$ denote learned encoder and decoder parameters. **Use:** nonlinear image or signal compression when one PCA subspace is inadequate; extra flexibility costs data and training effort.

# Related methods at a glance

| Practical need | Method to consider |
|---|---|
| Compress correlated continuous sensors | PCA |
| Represent sparse document–word counts | Truncated SVD |
| Model latent factors with isotropic noise | Probabilistic PCA |
| Allow feature-specific sensor noise | Factor analysis |
| Compress strongly nonlinear observations | Autoencoder |

**Rule:** start with a simple, transparent baseline and increase model complexity only when the task demands it.

# Practical PCA

CS5487 ▪ Semester A, 2026/27

# Training workflow

For a predictive task, PCA must be fitted **inside the training workflow**:

1. Split observations into training, validation and test sets.
2. Estimate any feature scaling from the training set.
3. Fit PCA directions using the training set only.
4. Transform validation and test observations using those fitted directions.
5. Choose $r$ by validation; reserve the test set for final evaluation.

**Centering, scaling and PCA are learned preprocessing steps, not fixed formulas.**

# PCA implementation

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95,
          svd_solver="full")

Z_train = pca.fit_transform(X_train)
Z_valid = pca.transform(X_valid)
Z_test = pca.transform(X_test)
```

- `fit_transform`: learn the training mean and directions, then encode training data.
- `transform`: encode new observations using that same fitted PCA.

**Use:** fit once and reuse in a predictive pipeline. For cross-validation, put scaling and PCA *inside* each training-fold pipeline.

# Data leakage

**Incorrect order**

all data $\rightarrow$ fit PCA $\rightarrow$ split.

The test set influences the fitted directions.

**Correct order**

split $\rightarrow$ fit on train $\rightarrow$ transform.

Validation/test sets are transformed only.

## Common mistake

Never use test observations when fitting means, scales or PCA directions.

# Leakage in numbers

Suppose the training observations of one feature are $1, 2, 3$. Training-only centering uses:

$$\bar{x}_{\text{train}} = \frac{1+2+3}{3} = \boxed{2}.$$

Now suppose the held-out test value is $100$. Incorrectly including it changes the mean to:

$$\bar{x}_{\text{all}} = \frac{1+2+3+100}{4} = \boxed{26.5}.$$

A training value $1$ would change from centered value $-1$ to $-25.5$. **Meaning:** information from the test set has changed the training representation; the same leakage can alter PCA directions and the claimed test performance.

# Conditions favoring PCA

PCA is a strong baseline when:

- Observations lie near one global linear subspace.
- Squared Euclidean reconstruction error reflects the task.
- Large-variance directions mainly represent useful signal.

**Examples:**

- Compress many correlated spectral bands before exploration.
- Reduce shared sensor variation before a simpler prediction model.
- Reconstruct normal operating patterns to inspect residuals.

**Check:** retain enough variance and validate the real task outcome.

# PCA limitations

PCA can be a poor representation when:

- The useful structure is curved rather than one flat subspace.
- An important class signal has low variance and gets discarded.
- Arbitrary feature units dominate the covariance.
- Outliers distort the principal directions.

**Example:** image brightness may vary much more than a subtle defect signal. High-EVR components need not be the best inputs for defect detection.

**The same representation trade-offs appear in modern Transformers, search systems, and neural autoencoders.**

# Modern applications

CS5487 ▪ Semester A, 2026/27

# Transformer activation analysis

A Transformer produces a hidden-state vector for each token or image patch. **Example:** collect $2{,}000$ token states, each with $768$ features, from one model layer:

$$\boldsymbol{H} \in \mathbb{R}^{2000\times 768} \xrightarrow{\text{center + PCA}} \boldsymbol{Z} \in \mathbb{R}^{2000\times 2}.$$

- Plot the $2$ PCA coordinates and color points by token group or input domain.
- Compare layers to inspect how their hidden-state representations change.
- Use held-out examples to check whether observed patterns are stable.

**Purpose:** understand or debug representations inside a trained model. **Distinction:** this PCA is an analysis step, not the Transformer's attention calculation.

# Compact embeddings for search

Semantic search systems may store a large vector for each document or product. **Illustrative example:** $1{,}000{,}000$ embeddings, $768$ coordinates each, stored as float32.

| Representation | Coordinates per item | Total vector storage |
|---|---|---|
| Original embeddings | $768$ | $3.072$ GB |
| PCA embeddings | $128$ | $0.512$ GB |

about $6\times$ less vector storage

**Use:** smaller vector indexes and less memory for similarity search. **Check:** compressed embeddings can change nearest neighbors; compare retrieval recall on queries. The shared PCA basis adds a small storage cost.

# Low-rank Transformer layers

SVD can approximate a large Transformer projection matrix by two smaller matrices.
**Illustrative example:** a layer with $\boldsymbol{W} \in \mathbb{R}^{1024\times 1024}$ is approximated at rank $r = 64$:

$$\boxed{\boldsymbol{W} \approx \boldsymbol{A}\boldsymbol{B}}, \qquad \boldsymbol{A} \in \mathbb{R}^{1024\times 64},\ \boldsymbol{B} \in \mathbb{R}^{64\times 1024}.$$

| Stored weights (biases excluded) | Count |
|---|---|
| Original $\boldsymbol{W}$ | $1{,}048{,}576$ |
| Two low-rank factors | $131{,}072$ |

**Benefit:** $87.5\%$ fewer weights for this matrix. **Limit:** approximation error can harm model outputs; validate accuracy and runtime. Weight compression by SVD is distinct from PCA of activations.

# Autoencoder reconstruction

An autoencoder learns an encoder and decoder instead of fixing the directions with PCA.
**Example:** a $28 \times 28$ grayscale image has $784$ pixel values. Use a $32$-dimensional latent code:

$$\underbrace{\boldsymbol{x} \in \mathbb{R}^{784}}_{\text{input}} \xrightarrow{f_\theta} \underbrace{\boldsymbol{z} \in \mathbb{R}^{32}}_{\text{code}} \xrightarrow{g_\phi} \underbrace{\hat{\boldsymbol{x}} \in \mathbb{R}^{784}}_{\text{reconstruction}}.$$

Train the maps to minimize average squared reconstruction error:

$$\mathcal{L}_{\text{rec}} = \frac{1}{n} \sum_i \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|_2^2.$$

**Use:** learn compact, potentially nonlinear image or sensor representations. **Compare:** PCA is a linear reconstruction baseline; the autoencoder may capture more complex variation but needs training and held-out evaluation.

# Masked reconstruction in ViTs

Masked autoencoders (MAEs) train vision models to recover image content without class labels. **Example:** a $224 \times 224$ image with $16 \times 16$ patches gives $14 \times 14 = 196$ patches.

$$\boxed{196 \text{ patches} \longrightarrow 49 \text{ visible} + 147 \text{ masked}}$$

This masks $75\%$ of the patches.

1. A Vision Transformer encoder processes the visible patches.
2. A decoder predicts pixels in the missing patches.
3. Reconstruction error on masked patches provides a training signal.

**Use:** pretrain visual representations from unlabeled images for later recognition tasks.
**Connection:** PCA and MAEs both learn from data structure, but MAEs use nonlinear context-based reconstruction, not PCA eigenvectors.

# Week 8 summary

1. **Geometry:** project centered data onto maximum-variance directions.
2. **Reconstruction:** discarded eigenvalues quantify squared information loss.
3. **Computation:** SVD computes principal directions and low-rank approximations.
4. **Practice:** apply PCA to compression, denoising, search, and representation diagnostics.
5. **Modern models:** connect SVD to Transformer compression and reconstruction to autoencoders.

$$\boxed{\text{center} \longrightarrow \text{project} \longrightarrow \text{reconstruct or predict}}$$

**Before deployment:** validate predictive quality, scaling, retained dimension, and data leakage.

# PCA turns covariance structure into useful low-dimensional coordinates

Questions and discussion